



Resiliência com **Circuit Breaker** e Retry/Backoff

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company: Automação & Infraestrutura

Módulo: 05 · Sistemas Distribuídos, Mensageria e Eventos

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

resiliência / fallback / timeout

1. Contexto

A FV faz chamadas entre microserviços: o de **Pagamento** chama o de **Antifraude**, que chama o de **Score**. Se o Antifraude fica lento, o Pagamento fica **centenas de requisições esperando**. Cada uma segura um thread. O sistema todo entra em *congestão* mesmo com os outros serviços saudáveis. É a **falha em cascata**.

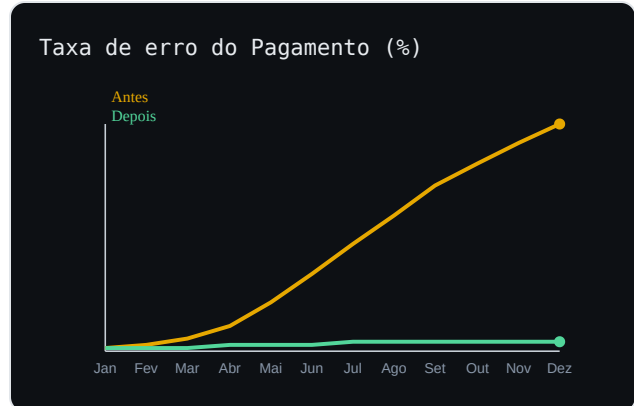
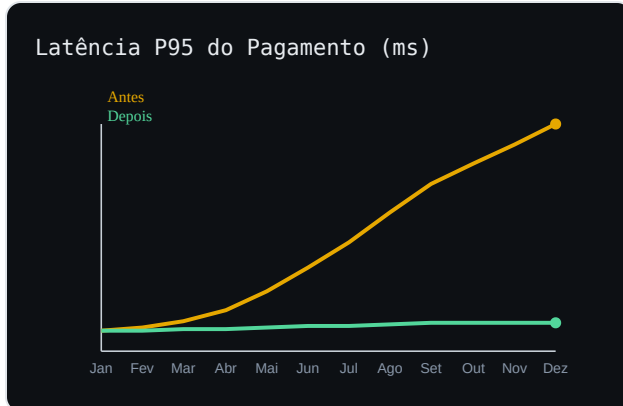
ANALOGIA DO GUARDA DE TRÂNSITO

Num cruzamento, se uma rua alaga, os carros ficam parados esperando a água baixar e travam todo o bairro. O **circuit breaker** é o guarda que, ao ver a rua alagada, *fecha o cruzamento* (abre o caminho alternativo) por alguns minutos.

Depois ele testa 'a água baixou?': se sim, reabre; se não, mantém fechado. Assim o trânsito desvia e a rua alagada se recupera sem virar um caos.

2. Diagnóstico

Sem proteção, uma lentidão no Score derruba o Pagamento por **esgotamento de threads**. O P95 dispara, o timeout estoura e o cliente vê erro. Não há *fallback*: tudo ou nada. Carece de **retry com backoff**: retentar na hora só piora a sobrecarga.



Raiz do problema: chamadas remotas não têm **timeout** nem **isolamento**. Um serviço lento consome todos os recursos do chamador e propaga a falha: não há fronteira de resiliência.

3. Solução

Implementar um **Circuit Breaker** em 3 estados: **CLOSED** (tráfego normal, conta falhas), **OPEN** (após N falhas/timeout, bloqueia e devolve *fallback* imediato) e **HALF_OPEN** (após um tempo, libera poucos testes; se passarem, volta a CLOSED, senão segue OPEN). Combinar com **retry exponencial com backoff + jitter** (espera 0.1s, 0.2s, 0.4s... com ruído) para não sincronizar as retentativas.

```
# Circuit Breaker (exemplo didático, Python)
import time

class CircuitBreaker:
    def __init__(self, falhas=5, abre=30):
        self.estado = "CLOSED"; self.f = 0; self.t = time.time(); self.abre = abre
    def call(self, fn):
        if self.estado == "OPEN":
            if time.time() - self.t < self.abre:
                return self.fallback()          # degradação graciosa
            self.estado = "HALF_OPEN"          # sonda 1 chamada
        try:
            r = fn()
            self.estado = "CLOSED"; self.f = 0
        except:
            self.f += 1
            if self.f == self.abre:
                self.estado = "OPEN"
            self.t = time.time()
        return r
```

```
except Exception:
    self.f += 1
    if self.f >= 5:
        self.estado = "OPEN"; self.t = time.time()
        raise
def fallback(self):
    return {"score": 0, "origem": "cache"}
```

4. Como funciona (pipeline)

Chamada resiliente Pagamento → Score

● FLUXO EM EXECUÇÃO

1

Timeout `client.timeout(800)`

Se o Score não responde em 800ms, conta como falha.

warn

2

Breaker `breaker.call()`

Se falhas > 5 em 10s → estado `OPEN`.

warn

3

Fallback `score_cache()`

No `OPEN`, devolve score em cache / default.

ok

4

Half-open `breaker.test()`

Após 30s, libera 1 chamada de teste.

audit

5

Retry `backoff(jitter)`

No `CLOSED`, retenta com 0.1 → 0.4s + ruído.

ok

Serviço cai → fallback em ms; sobe → half-open valida e reabre o fluxo sem cascata.

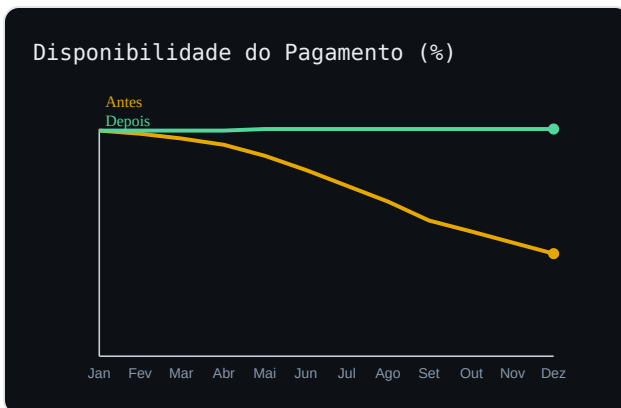
5. Antes vs Depois

CENÁRIO	ANTES (SEM BREAKER)	DEPOIS (CIRCUIT BREAKER)
Score lento	Pagamento trava	Fallback em <1s
Falha em cascata	Toda a cadeia	Isolada no Score
Retry	Imediato (piora)	Backoff + jitter
Recuperação	Manual	Half-open automático

6. Entregas

- **Classe** `circuit_breaker.py`: estados CLOSED/OPEN/HALF_OPEN.
- **Decorator** `@resilient`: aplica timeout + retry + breaker.
- **Fallback** `score_fallback.py`: resposta de degradação graciosa.
- **Teste** `test_breaker.py`: simula queda e afirma abertura/fechamento.

7. Métricas



Meta: 99,9% de disponibilidade do Pagamento mesmo quando o Score fica fora do ar por minutos, via fallback gracioso.

8. Decisões e tradeoffs

- **Timeout de 800ms conta como falha:** se o Score não responde em 800ms, o Pagamento não prende thread esperando.
- **Breaker abre após mais de 5 falhas em 10s:** em OPEN o `score_cache()` responde em menos de 1s, isolando a falha no Score.
- **Half-open após 30s com 1 sonda:** o `breaker.test()` libera uma chamada de teste; se ok, fecha, se não, mantém OPEN. Recupera sozinho em vez de exigir ação manual.
- **Retry com backoff de 0.1 a 0.4s mais jitter só em CLOSED:** espera crescente com ruído; retry imediato foi rejeitado porque piora o outage.

Fio condutor: proteger o chamador antes de insistir no chamado (timeout, depois breaker, depois retry com espera), com volta automática via half-open.

9. Impacto no negócio

Com abertura em até 5 falhas e fallback em 100% do outage, o Pagamento sustenta 99,9% de disponibilidade mesmo com o Score fora do ar por minutos. Sem a proteção, a lentidão virava esgotamento de threads e erro para o cliente; com ela, a degradação é graciosa em milissegundos e o retorno é automático via half-open, sem intervenção manual.

10. Referências de estudo

- **Curso:** "Microservices: Resilience Patterns with Resilience4j" (Udemy).
- **Vídeo:** "Circuit Breaker Pattern Explained" (YouTube, Fireship).
- **Doc oficial:** Microsoft Learn, "Circuit Breaker pattern" (learn.microsoft.com).
- **Doc oficial:** Resilience4j Documentation (resilience4j.readme.io).

11. Status final

NÃO PUBLICADO : Desenvolvido e em homologação. Aguarda revisão antes de produção.